TECHNISCHE UNIVERSITÄT
CHEMNITZ

Department of Electrical Engineering and Information Technology

Chair of Measurement and Sensor Technology

Project Documentation

"Project Lab Embedded Systems"

Group: 29

Members: Chaitanyarani Veeranna Kiragi

Prasad Ganesh Pai

Supriya Madenahalli Rajashekaraiah

Tayiba Fathima Syed

Project: Design and Simulation of an Automotive Battery Pack Moisture Monitoring and Thermal Management System (BTMS)

Date: 2026-08-12

Table of Contents

1. Abstract **1**

2. Member Responsibilities **1**

3. Introduction **2**

3.1 Background 2

3.2 Problem Statement 2

3.3 Motivation 2

3.4 Objectives 2

3.5 Scope 3

4. Literature Review **3**

4.1 Moisture Monitoring and Capacitive Sensing 3

4.2 Signal Conditioning and Calibration 3

4.3 Diagnostics, Safety, and Predictive Concepts 3

5. Project Overview **4**

5.1 Initial Concept vs. Final Architecture 4

5.2 Development Timeline 5

6. System Requirements **6**

6.1 Functional Requirements 6

6.2 Non-Functional Requirements 6

6.3 Assumptions and Constraints 7

7. System Architecture **7**

7.1 Overview 7

7.2 Input Layer 7

7.3 Analog Layer 7

7.4 Software Layer 7

7.5 Actuation Layer 8

7.6 Summary 8

8. Hardware Design **9**

8.1 Hardware Architecture 9

8.2 Capacitive Humidity Sensor Model 9

8.3 Analog Front-End 9

8.4 Comparator-Based BTMS Controller 10

8.5 Components Selection and Justification 11

8.6 Design Consideration 12

9. Software Design **12**

9.1 Sensor Interface and Peak-to-Peak Voltage 12
9.2 Temperture Compensation 12
9.3 Calibration 13
9.4 Calibration Validation 14
9.5 Sensor Health Index and Diagnostic Confidence 14
9.6 Diagnostics 14
9.7 Smart battery Risk Score 14
9.8 Battery Health and Safety Index 14
9.9 Moisture, Corrosion and Condensation Risk 14
9.10 Protection Stage and Recommended Action 15
9.11 Thermal Management 15
9.12 Predictive Maintenance 15
9.13 GUI-Dashboard 16
9.14 Software Design Considerations 16
10. System Execution 16
10.1 Environmental Simulation 16
10.2 Sensor Modelling 16
10.3 Analog Signal Conditioning 17
10.4 Python ECU Processing 17
10.5 Decision Logic and Thermal Management 17
10.6 System Feedback 17
10.7 System Timing 17
11. Results and Evaluation 18
11.1 LTspice Verification 18
11.2 BTMS Hardware Verification 19
11.3 Python Calibration and Compensation Verification 21
11.4 Battery Assessment and Diagnostics Verification 23
11.5 Verification Summary 24
12. Engineering Challenges and Solutions 25
13. Verification, Validation and Test Plan 26
13.1 Verification Strategy 26
13.2 Validation Strategy 26
13.3 Requirement Traceability Matrix 27
13.4 Test Cases 27
14. Discussion and Design Rationale 30

14.1 Tool Choices 30
14.2 Hardware/Software Partitioning 30
14.3 Comparison with a Conventional BMS 30
14.4 Observation: Sensor Health Index Does Not Distinguish Fault Types 31
15. Limitation and Risk Assessment 31
15.1 Limitations 31
15.2 Risk Assessment 32
16. Future Work and Industrial Relevance 32
16.1 Near-Term Priorities 32
16.2 Path to Physical Hardware and AUTOSAR 33
16.3 Further Work 33
17. Conclusion 33
Bibliography 34
Appendix A Abbreviations 35
Appendix B Symbols 36
Appendix C Complete Equations Summary 37

1. Abstract

Electric vehicle battery packs are increasingly monitored for voltage, current, and temperature, yet moisture ingress inside the enclosure is rarely given a dedicated monitoring path of its own, despite its role in accelerating corrosion, degrading insulation, and raising leakage-current risk. This report presents a simulation-based Battery Pack Moisture Monitoring and Thermal Management System (BTMS) that pairs a capacitive humidity sensor model with an LTspice analog front-end and a Python-based electronic control unit responsible for calibration, temperature compensation, fault diagnostics, smart risk-based battery assessment, predictive maintenance, and thermal decision logic.

The hardware side is modelled entirely in LTspice: sensor excitation, capacitance-to-voltage conversion, gain and filtering, comparator-based threshold detection, and a MOSFET-driven BTMS controller with dedicated heating, cooling, and over-current protection stages. The software side, implemented in Python, converts the exported peak-to-peak voltage into a temperature-compensated, calibrated humidity value, screens it through diagnostics and a sensor health index, combines multiple factors into a weighted smart risk score, and issues thermal-management and protection-stage commands back toward the simulated actuators closing the loop between analog sensing and digital decision-making without any physical prototype.

The report documents the system as it currently stands, the reasoning behind the major design decisions, the concrete engineering problems encountered while developing the analog front-end and the comparator/MOSFET thermal stages, and how the resulting software structure could be carried forward onto an AUTOSAR-style automotive architecture. Chapter 9 presents simulation results captured directly from LTspice and the Python ECU, including real fault-injection logs; three specific measurement gaps identified while assembling those results (load-current probing, an over-current trip capture, and a Battery Health vs. Humidity sweep) are stated explicitly in Chapter 14 rather than filled with placeholder data.

2. Member Responsibilities

This project was carried out by a team of four. The table below is left for the team to complete, recording each member's primary area of responsibility and their specific contribution to the work described in this report.

Name	Contribution Summary
Chaitanyarani Kiragi	Python ECU modules (Partial) , BTMS actuation hardware (comparator/MOSFET) & fault scenarios in Ltspice, Sensor output (VPP) validation, Dashboard GUI (Partial).
Prasad Ganesh Pai	System architecture & LTspice AFE circuits, Simulation setup, Dashboard GUI (Partial), Module integration, CSV logging & testing, Python ECU modules (Partial).

Name	Contribution Summary
Supriya Madenahalli	Literature review & requirements sections, Result graphs & validation tables, Assistance in LTSpice, Presentation slide deck, Test case validation.
Tayiba Fathima Syed	Test-case execution, Formatted report & diagrams, log & screenshots, Assistance in LTSpice, Proof reading, Demo run-through

3. Introduction

3.1 Background

Battery management in electric vehicles has matured considerably around voltage, current, and temperature monitoring, since these parameters govern charging behaviour and thermal-runaway risk. Moisture inside a battery enclosure is a comparatively quieter threat: it accumulates gradually through damaged seals, manufacturing tolerances, or condensation following rapid temperature swings, and by the time its effects corrosion, reduced insulation resistance, elevated leakage current becomes visible through electrical measurements alone, the damage is often already underway. This motivates monitoring humidity directly rather than inferring it indirectly.

3.2 Problem Statement

Humidity sensing is rarely built into low-cost battery monitoring systems, and evaluating a new monitoring strategy on real hardware typically requires a sensor, a purpose-built PCB, and laboratory test equipment before any useful data can be collected. This project instead asks how far a purely simulation-based platform an LTspice analog front end paired with a Python-based processing layer can go in demonstrating the same monitoring, diagnostic, and thermal-management behaviour that a physical system would need, without incurring that hardware cost up front.

3.3 Motivation

The intent from the outset was broader than a single humidity sensor interface. Automotive electronic control units rarely perform one function in isolation; they calibrate, compensate, diagnose, and make decisions as an integrated chain. Building the project around that same chain signal conditioning, calibration, temperature compensation, diagnostics, functional safety, predictive maintenance, and thermal decision logic gives a more realistic picture of what an automotive-style ECU actually does, and keeps the simulation cheap enough to iterate on quickly before any hardware commitment is made.

3.4 Objectives

The project set out to:

- Model a capacitive humidity sensor mathematically, including its dependence on temperature.
- Design an LTspice analog front-end for signal acquisition and conditioning.
- Simulate a representative range of humidity and temperature operating conditions.
- Implement calibration, linearization, and temperature compensation in Python.
- Implement diagnostic logic capable of flagging sensor faults.
- Implement a functional-safety state machine with a defined fallback strategy.

- Implement predictive-maintenance indicators for early warning of moisture ingress.
- Design comparator-based heating and cooling controllers with MOSFET driver stages.
- Demonstrate a working hardware/software co-simulation between LTspice and Python.

3.5 Scope

Everything described in this report is a simulation. No sensor, PCB, microcontroller, or battery pack was built or tested physically. LTspice models the analog subsystem; Python emulates the ECU. The battery and actuator models used are deliberately simplified, but the overall architecture is structured to resemble a practical automotive environmental-monitoring system closely enough to serve as a starting point for a future hardware implementation.

4. Literature Review

4.1 Moisture Monitoring and Capacitive Sensing

Conventional battery management systems supervise voltage, charging current, cell balance, and temperature, but rarely humidity — moisture ingress instead tends to surface only once its consequences (corrosion, reduced insulation resistance, elevated leakage current) are already measurable elsewhere. This project treats humidity as a first-class monitored variable instead. Capacitive humidity sensors are the common choice for this in automotive and industrial electronics: absorbed moisture raises a dielectric layer's constant, which raises capacitance per the parallel-plate relationship C = eA/d (e = dielectric constant, A = plate area, d = plate separation) — the basis of the sensor model used throughout this project.

4.2 Signal Conditioning and Calibration

Since capacitance cannot be read directly by a digital control unit, a capacitance-to-voltage front-end is required first — typically a charge amplifier, a gain stage, and a low-pass filter, the same structure used here (Chapter 8). Because sensor outputs are rarely linear, a calibration stage (commonly a polynomial or linear fit between measured voltage and known humidity) linearizes the response, and a separate compensation step corrects for temperature drift on top of that.

4.3 Diagnostics, Safety, and Predictive Concepts

Reliable sensing also requires recognizing when a reading should not be trusted: out-of-range values, spikes, a stuck signal, or unexplained drift are the four fault categories commonly used to flag misbehaviour, and are the ones adopted here (Chapter 10). A functional-safety layer typically tracks a small number of discrete states (e.g. OK/DEGRADED/FAULT/CRITICAL) with a defined fallback; predictive maintenance extends this by tracking trend over time rather than only the instantaneous value, to estimate how long a system has before a critical threshold is reached. Both concepts inform this project's design, with the specific implementation status of each noted in Chapter 9.

5. Project Overview

The system was not implemented as a single design pass. It was built up in phases, each adding one function on top of a working base, which kept debugging manageable and made it possible to verify each addition in isolation before moving on. This section summarizes that development sequence; the engineering problems encountered along the way are treated in more depth in Chapter 12.

5.1 Initial Concept vs. Final Architecture

It is worth contrasting where the project started against where it ended up, since the gap between the two is a good measure of how much the scope grew during development. The original objective, stated at the start of the project, was considerably narrower than the final result:

Original workflow: Humidity -> Humidity Sensor -> Analog Front-End (LTspice) -> Measured VPP -> Python Calibration -> Estimated Humidity.

At that stage, the project consisted only of an LTspice analog front-end, a Python humidity estimation script, and a single calibration equation — nothing else. The final architecture, shown in Figure 6.1 in the next chapter, instead runs the sensor signal through temperature compensation, calibration, diagnostics and fault detection, a smart battery-health assessment engine, parallel thermal-management and predictive-maintenance paths, a combined decision-logic and GUI dashboard layer, and finally a hardware BTMS controller that drives real cooling/heating actuator paths in simulation. What began as a single calibration equation grew into a nine-stage processing pipeline plus a dedicated hardware controller, which is the change documented phase-by-phase below.

Phase 1 — Capacitive Humidity Sensor Model

The project began with a parameterized capacitance model, C(RH,T) = C0(1 + alpha*RH + beta*(T-T0)), so that humidity and temperature could be varied systematically in simulation rather than fixed at a single operating point.

Phase 2 — LTspice Analog Front-End

A charge amplifier converted the sensor's capacitance into a measurable voltage, followed by a gain stage and a low-pass filter, giving a clean analog signal proportional to humidity.

Phase 3 — Calibration

A second-order polynomial fit (RH = aV^2 + bV + c) was used to convert the front-end's voltage output into a linear humidity estimate, since the raw sensor response is not linear across its working range.

Phase 4 — Temperature Compensation

A correction term, RH_corrected = RH_raw - k(T - T0), was applied after calibration to remove the temperature-dependent offset in the calibrated reading, using k as an experimentally chosen compensation coefficient and T0 as the sensor's reference temperature.

Phase 5 — Diagnostics

Range, spike, stuck-signal, and drift checks were added to flag sensor misbehaviour before a faulty reading could reach the rest of the pipeline.

Phase 6 — Risk-Based Battery Assessment

Rather than a single fixed humidity threshold, a weighted risk score combining humidity, temperature, sensor health, and diagnostic confidence was introduced, feeding staged protection actions. A fully formal functional-safety state machine (OK/DEGRADED/FAULT/CRITICAL, with a proven fallback guarantee) was scoped for this phase but was not completed; it remains future work, discussed in Chapter 15.

Phase 7 — Predictive Maintenance

Trend analysis (dRH/dt), a time-to-critical estimate ((70 - RH)/slope), and a normalized health index (1 - RH/70) were added on top of the diagnostic-filtered signal, to flag a rising moisture trend before it reaches the critical threshold.

Phase 8 — Automotive Decision Logic

A decision-logic stage converted the risk assessment and predictive indicators into concrete thermal-management commands, closing the loop between diagnosis and action.

Phase 9 — Battery Thermal Management System (BTMS)

The final phase added a comparator-based thermal controller: separate heating and cooling comparators, each driving a MOSFET stage, so that the software's decisions could be exercised against a simulated actuator load rather than remaining purely numerical.

5.2 Development Timeline

Table 5.2 summarizes the phase sequence described above.

Phase	Focus
1	Problem analysis and requirements
2	Capacitive sensor modelling
3	LTspice analog front-end
4	Signal conditioning
5	Calibration
6	Temperature compensation
7	Diagnostics
8	Risk-based battery assessment
9	Predictive maintenance
10	Automotive decision logic

Phase	Focus
11	Comparator-based BTMS (heating/cooling)
12	Final system integration

Table 5.2 — Development phase sequence

6. System Requirements

The following requirements were defined before implementation and guided the design of both the LTspice hardware model and the Python software design.

6.1 Functional Requirements

ID	Requirement
FR-01	Simulate a capacitive humidity sensor for battery pack monitoring.
FR-02	Convert humidity-dependent capacitance into an analog voltage.
FR-03	Amplify and filter the analog sensor signal.
FR-04	Export the processed signal for software analysis.
FR-05	Convert voltage into a calibrated humidity value.
FR-06	Apply temperature compensation to improve measurement accuracy.
FR-07	Detect abnormal sensor behaviour (out-of-range, spike, stuck, drift).
FR-08	Classify system operation into functional-safety states.
FR-09	Estimate future moisture ingress using predictive-maintenance logic.
FR-10	Assess overall battery condition from the corrected humidity value.
FR-11	Drive comparator-based heating and cooling controllers from software decisions.

Table 6.1 — Functional Requirements

FR-08 (functional-safety state classification) remains an open requirement at the time of writing; see the objective-achievement status in Table 1.1 and the discussion in Chapters 15-16.

6.2 Non-Functional Requirements

ID	Requirement
NFR-01	The system shall be fully realizable in simulation, without physical hardware.
NFR-02	The analog and software subsystems shall be independently testable.
NFR-03	The architecture shall be modular enough to support future porting to embedded/AUTOSAR targets.

ID	Requirement
NFR-04	Processing stages shall execute in a fixed, deterministic order.

Table 6.2 — Non-Functional Requirements

6.3 Assumptions and Constraints

The battery and actuator models are simplified representations rather than characterized physical devices; comparator thresholds and MOSFET parameters are chosen for demonstrably stable switching in simulation rather than tuned against a specific commercial part; and the Python side treats the LTspice-exported signal as though it had already passed through a real ADC, without modelling ADC quantization or sampling-rate effects explicitly.

7. System Architecture

7.1 Overview

Figure 7.1 shows the complete signal path: the environment (humidity and temperature) drives a simulated capacitive sensor; the LTspice analog front-end converts the resulting capacitance into a measured peak-to-peak voltage; the Python ECU applies temperature compensation and calibration to recover an estimated humidity value, runs diagnostics and fault detection on it, and feeds the result into a Smart Battery Health Assessment Engine. That engine's output branches into Thermal Management and Predictive Maintenance in parallel, which converge into a combined ECU decision-logic and GUI dashboard layer. Decision-logic output finally drives the BTMS controller — a comparator/MOSFET-based hardware stage — which switches the simulated cooling-fan and heating-pad drivers, closing the loop back toward a safe enclosure condition.

7.2 Input Layer

The input layer is the simulated capacitive humidity sensor, with temperature supplied as a runtime input value (from a user setting or a temperature-sensor model) rather than a fixed constant. Both feed the analog front-end as a software/SPICE-level model rather than a physical part.

7.3 Analog Layer

The analog front-end (detailed in Chapter 8) performs sensing and signal conditioning, producing the measured peak-to-peak voltage (VPP) that stands in for what a real ADC on physical hardware would digitize.

7.4 Software Layer

The Python ECU (detailed in Chapter 9) performs temperature compensation, calibration, diagnostics and fault detection, battery-health/risk assessment, thermal decision logic, and

predictive maintenance, in that fixed order (see Chapter 14 for the current scope of the safety-related logic).

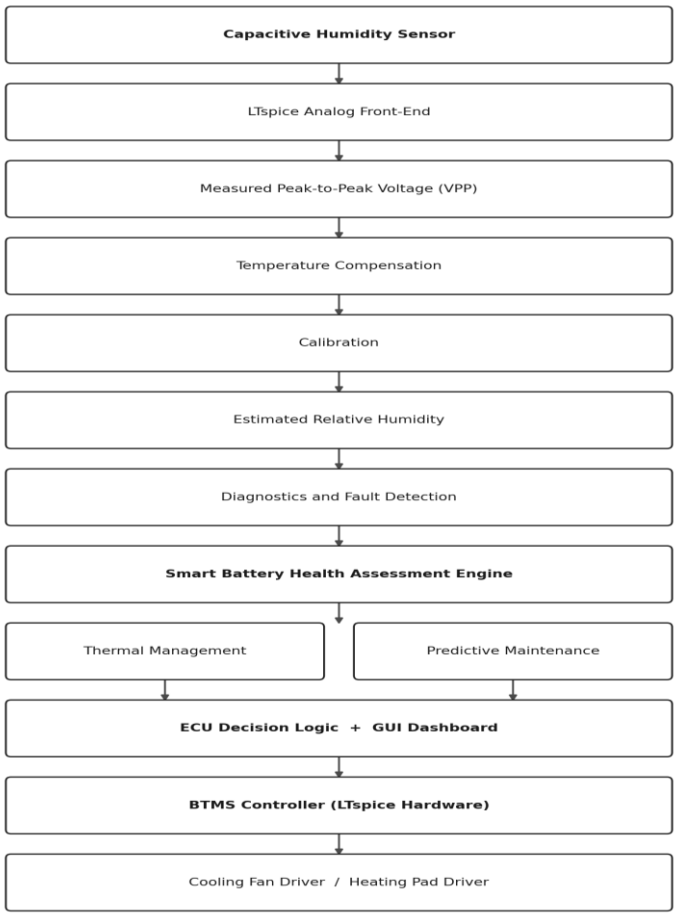


Figure 7.1 — Final end-to-end system pipeline (hardware/software co-simulation)

7.5 Actuation Layer

Decision-logic output drives the BTMS controller — comparator/MOSFET stages for cooling and heating respectively, plus an over-current protection comparator — each switching a simulated load that stands in for a fan, heater, or protection fuse. These stages are described in Chapter 8.

7.6 Summary

The architecture keeps hardware and software cleanly separated at the point where the LTspice-exported VPP value crosses into Python, which is also where a real ADC would sit in a physical implementation — a deliberate choice made to keep the simulation boundary in the same place a hardware/software boundary would naturally fall.

8. Hardware Design

This chapter describes the simulated analog hardware in detail. As stated in Chapter 1, nothing described here has been built physically; all values and behaviours come from the LTspice netlist.

8.1 Hardware Architecture

The hardware subsystem has two distinct halves that share the same LTspice environment but serve different purposes: a sensing/conditioning chain (sensor, charge amplifier, gain stage, low-pass filter) that produces the humidity-proportional voltage, and a thermal-actuation chain (comparators, MOSFET drivers, heating and cooling loads) that reacts to the ECU's decisions rather than to the sensor signal directly.

8.2 Capacitive Humidity Sensor Model

The sensor is represented as C(RH, T) = C0(1 + alpha*RH + beta*(T-T0)), with RH and T swept across the expected operating envelope rather than fixed at one point, so the front-end's response can be characterized across the full range in a single set of simulation runs.

8.3 Analog Front-End (AFE)

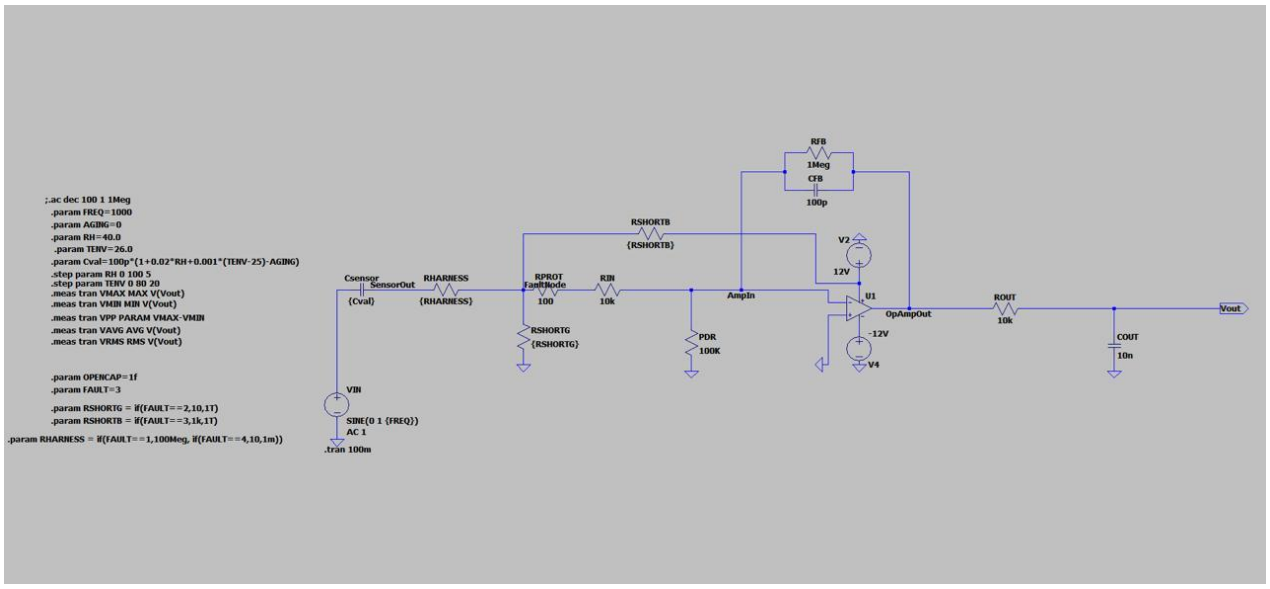


Figure 8.1 — LTspice analog front-end schematic

An AC excitation source drives the sensor capacitance; a charge amplifier converts the resulting current into a voltage; a gain stage sets the overall sensitivity; and a low-pass output filter removes high-frequency content before the signal is treated as the measured output. This is the stage exported to Python for all downstream processing.

8.4 Comparator-Based BTMS Controller

Rather than driving actuators directly from software, the decision-logic output is compared in LTspice against fixed voltage thresholds using comparator circuits one tuned for the cooling condition, one for the heating condition, and a third dedicated to over-current protection each producing a clean high/low output that switches a MOSFET stage. This keeps the actual switching decision at the hardware level, which is closer to how a discrete thermal-control circuit would behave on real hardware than a purely software-timed switch would be. In the current implementation, both driver paths are built onto a single BTMS Power Driver Board netlist, shown in Figure 8.2.

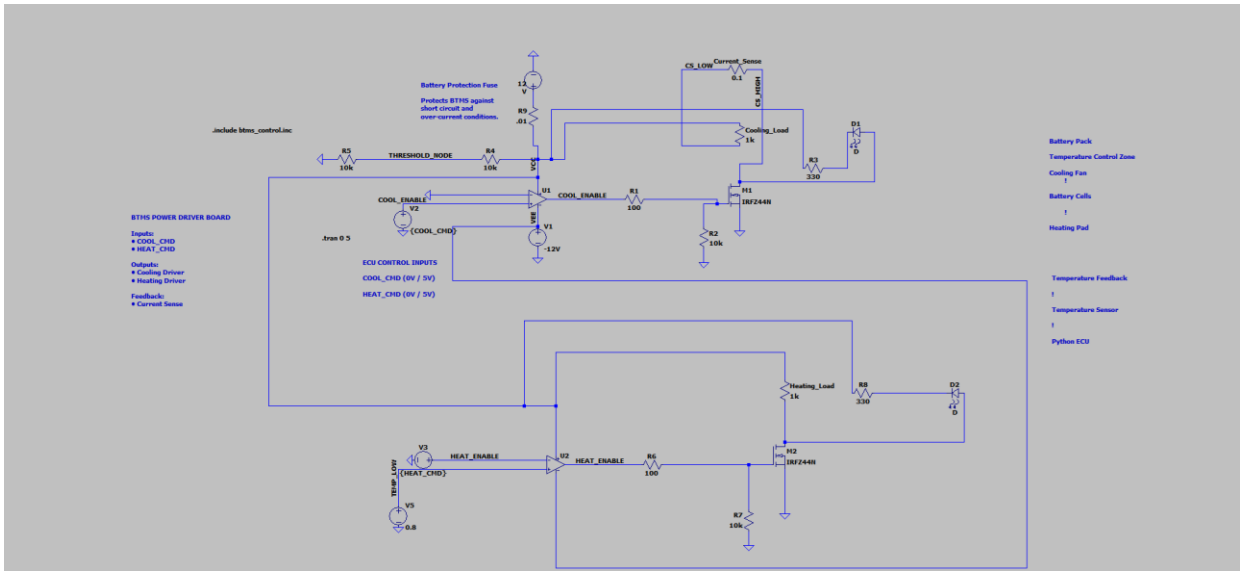


Figure 8.2 — BTMS Power Driver Board: cooling and heating driver circuits (as simulated)

8.4.1 Cooling Driver

The cooling side takes a COOL_CMD input, buffers it through comparator U1 as a COOL_ENABLE signal, and drives MOSFET M1 (IRFZ44N) through a 100 Ω gate resistor (R1), with a 10 kΩ pull-down (R2) holding the gate low whenever COOL_ENABLE is not actively asserted. M1 switches the 1 kΩ Cooling Load, with an indicator LED (D1) and its 330 Ω series resistor (R3) providing a visual on/off cue on the board.

8.4.2 Heating Driver

The heating side mirrors this structure: a HEAT_CMD input drives comparator U2, producing HEAT_ENABLE, which switches MOSFET M2 (IRFZ44N) through its own 100 Ω gate resistor (R6) and 10 kΩ pull-downs (R7), driving the 1 kΩ Heating Load with its own indicator LED (D2) and 330 Ω series resistor (R8).

8.4.3 Over-Current Protection

A current-sense resistor (0.1 Ω, labelled Current Sense in Figure 8.2, between nodes CS_HIGH and CS_LOW) sits in series with the shared 12 V supply feeding both loads, and a 0.01 Ω series element (R9) represents the battery-protection fuse ahead of it. A comparator monitoring the voltage across the sense resistor is intended to trip a distinct protection output once sensed

current exceeds the design's ~0.5 A threshold (Section 8.5), disabling the driver stage. A clean waveform capturing this trip event was not available at the time of writing; see Section 11.1 for the closest available evidence (gate-voltage confirmation without a clear current-trip capture) and Chapter 15 for this noted as an open verification item.

8.5 Component Selection and Justification

Component choices were driven by simulation stability and by being able to justify each value with a simple calculation, rather than by commercial sourcing, since no physical build was intended.

Component	Value	Purpose / justification
MOSFET	IRFZ44N (model)	Switches the cooling/heating loads; substituted for the generic LTspice NMOS model, which did not switch reliably under the higher simulated loads used here.
Gate resistor	100 Ω	Limits inrush current into the MOSFET gate during switching.
Gate pull-down resistor	10 kΩ	Prevents the gate from floating (and the MOSFET from switching on unintentionally) when the driving comparator output is not actively pulling it high or low.
Current-sense resistor	0.1 Ω	Produces a small, measurable voltage drop proportional to load current while keeping power loss low. At 0.5 A, V = I x R = 0.5 x 0.1 = 0.05 V.
Over-current comparator threshold	0.05 V	Matches the sense-resistor voltage at the intended trip current. By Ohm's law, I = V / R = 0.05 / 0.1 = 0.5 A, so the comparator trips at approximately 0.5 A of load current.
Fuse	0.01 Ω (simulated)	Represents a battery-protection fuse in series with the load, modelled with a low nominal resistance so it does not affect normal operation.

Beyond the thermal-actuation stage, the analog sensing chain uses operational amplifiers for the charge-amplifier and gain stages, chosen for their high input impedance, which suits the weak signal levels coming from the sensor; and comparators, rather than a microcontroller-based threshold check, for all three switching decisions (cooling, heating, over-current), since a comparator gives a deterministic, hardware-level trip point that does not depend on a software polling loop.

Table 8.1 — BTMS controller component values and justification

Component / stage	Role
Capacitive sensor model	Converts humidity (and temperature) into a simulated capacitance
Charge amplifier (op-amp)	Converts sensor capacitance into a proportional voltage
Gain stage	Sets overall front-end sensitivity

Component / stage	Role
Low-Pass filter	Removes high-frequency noise before the signal leaves the analog domain
Comparator (cooling path)	Switches the cooling MOSFET when the upper threshold is exceeded
Comparator (heating path)	Switches the heating MOSFET when the lower threshold is exceeded
Comparator (protection path)	Trips the protection output when sensed current exceeds ≈ 0.5 A

Table 8.2 — Simulated analog sensing and switching components

8.6 Design Considerations

The front-end's gain stage was tuned to produce a clearly resolvable output swing without introducing instability; comparator thresholds were chosen to emulate plausible thermal limits for a battery enclosure, with enough separation between the heating and cooling thresholds to avoid rapid on/off cycling; the current-sense resistor value was chosen specifically so that the intended trip current (≈ 0.5 A) lines up with a comparator threshold (0.05 V) that is easy to generate and verify in LTspice; and simplified resistive loads stand in for a physical fan or heater, since the switching behaviour rather than the thermal load itself is what this project is demonstrating.

9. Software Design

A Python-based Electronic Control Unit (ECU), organized as a series of specialized modules that each handle a single function and transfer its output to the next, handles all intelligence processing and decision-making while LTspice models the analog sensing and actuator hardware. Equations directly implemented in code and behavior represented as threshold-based decision logic instead of a single closed-form formula are distinguished below; both are acceptable, but the distinction is crucial for accurately describing the system.

9.1 Sensor Interface and Peak-to-Peak Voltage

The sensor interface calculates VPP = VMAX - VMIN after reading the output signal from the front-end. Since peak-to-peak voltage is less sensitive to DC offset and amplifier bias, it is preferred over a single instantaneous value. VPP is computed in Python and shown as "Measured VPP"—the beginning point for each downstream module; VMAX/VMIN are derived from the LTspice waveform.

9.2 Temperature Compensation

The recorded voltage is adjusted before being converted to humidity since the sensor output wanders with temperature: $V_{pp_corr} = V_{pp} - k (T - T_{ref})$. T is a runtime input; $T_{ref} = 25°C$ is the calibration reference temperature (Section 8.3); and $k = 0.0002525$ V/ °C is a sensor-specific, rather than universal, coefficient found during sensor characterisation. Because output voltage increases with temperature even at constant humidity, the adjustment is

subtracted rather than added because the excess must be eliminated before it is mistakenly interpreted as increased humidity. Python: Vpp_corrected = (temperature - 25) * Vpp - 0.0002525.

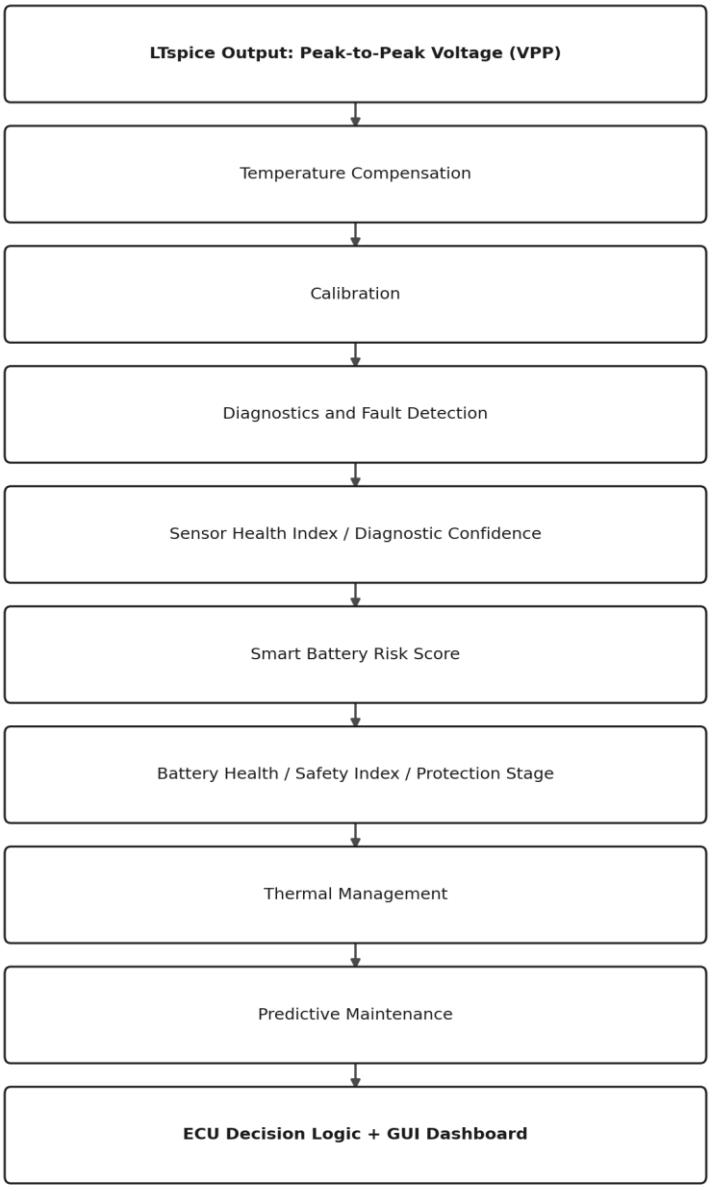


Figure 9.1 — Python ECU processing pipeline

9.3 Calibration

A linear fit is used to convert the corrected voltage to estimated humidity: RH = (VPP_corr - 0.8677) / 0.0061. The sensitivity (0.0061 V per %RH) was acquired by fitting a line thru observed VPP values at known humidity levels during LTspice characterization; it was neither assumed or taken from a datasheet. The offset (0.8677 V) corresponds to 0% RH after correction. The "Estimated Humidity" value on the dashboard is generated by this equation.

9.4 Calibration Validation

Before being trusted, an estimated humidity value is compared to a reasonable range (0–100% RH) and the anticipated sensor voltage range; readings outside of these ranges are marked invalid instead of being passed downstream.

9.5 Sensor Health Index (SHI) and Diagnostic Confidence

Both are implemented as decision algorithms that accumulate a penalty from several diagnostic criteria rather than a single equation, yet they both report a score between 0 and 100%. Confidence = 100 - Error (error reflects calibration variation, instability, or aberrant readings); SHI = 100 - Penalty (penalty increases with abnormal readings, decreased confidence, or discovered defects). Both are displayed on the dashboard and contribute to the Smart Risk Score (Section 9.7).

9.6 Diagnostics

Independently of SHI, four checks run on the compensated signal:

- Range: voltage/humidity outside a physically plausible band.
- Spike: sample-to-sample change exceeding an expected threshold.
- Stuck signal: near-zero variance over a recent window.
- Drift: slow, continuous deviation inconsistent with expected behaviour.

Each contributes to a diagnostic status (HEALTHY/WARNING/FAULT), a trouble code, and a severity level, shown alongside the numeric diagnostic confidence.

9.7 Smart Battery Risk Score

Earlier risk assessment used one fixed threshold (RH>90% = critical); the current implementation combines five weighted contributions: Risk = 0.40 H + 0.20 T + 0.15 SHI + 0.15 DC + 0.10 TM. Each factor is first scored 0-100 via threshold logic before weighting. The weights are engineering assumptions, not fitted values — humidity dominates (most directly tied to moisture ingress/corrosion), temperature is secondary, and sensor health/diagnostic confidence discount the score when the underlying measurement is untrustworthy. Weights sum to 1.0.

9.8 Battery Health and Safety Index

BatteryHealth = 100 - 0.5 x RiskScore; SafetyIndex = 100 - RiskScore. The 0.5 factor is deliberate: without it, maximum risk would drive battery health to 0%, overstating what a moisture/thermal score alone can conclude about actual battery condition. Neither equation measures true battery capacity — both are interpretive indicators built on the risk score.

9.9 Moisture, Corrosion, and Condensation Risk

Three further indicators, all threshold-based rather than closed-form: Moisture Risk (Low/Medium/High/Critical bands on RH); Corrosion Risk (rises with combined humidity and temperature); Condensation Risk (dew-point-style logic from both, reported LOW/HIGH).

9.10 Protection Stage and Recommended Action

The risk score maps to one of four protection stages (Monitoring, Charging Limitation, Protection Active, Immediate Inspection/Shutdown) via threshold logic. This determines Charging Permission (YES/NO) and an auto-generated Recommended Action (e.g. "Reduce Charging", "Service Battery Pack"). A short Root Cause Analysis lists the specific contributing factors (e.g. high humidity, poor sensor health) so the dashboard explains what is driving the assessment, not just a number.

9.11 Thermal Management

Fixed threshold logic on temperature, humidity, and battery status:

- Below 15 degC -> Heating mode
- 15-35 degC -> Standby / normal
- Above 35 degC -> Cooling mode

These are not sensor-derived; rather, they are engineering decisions for a reasonable safe range. A predicted temperature and fan speed maps from the same decision are also shown. The BTMS controller hardware is powered by the output of this step (Chapter 8). The dashboard's Thermal Management panel correctly enters Heating mode when the temperature is low, as seen in Figure 9.2.

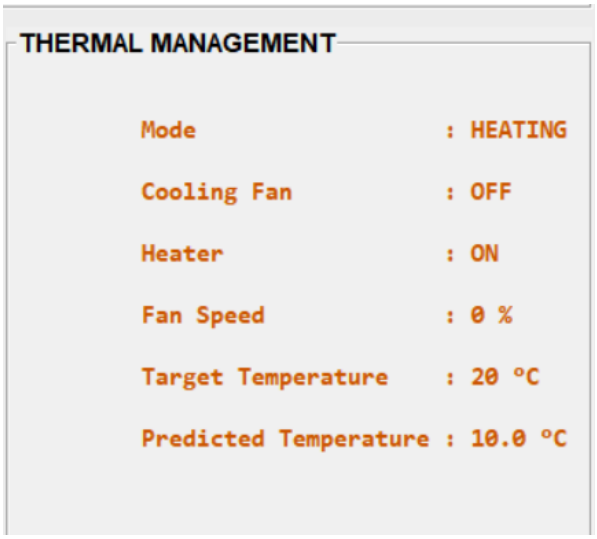


Figure 9.2 — Thermal Management panel: Heating mode correctly selected below the 15 deg°C threshold

9.12 Predictive Maintenance

Tracks sensor health over a rolling history rather than a single reading: current Sensor Health, Average Health, Health Trend, and a Maintenance Recommendation once the trend crosses an internal threshold — historical averaging and trend analysis, not one formula.

9.13 GUI-Dashboard

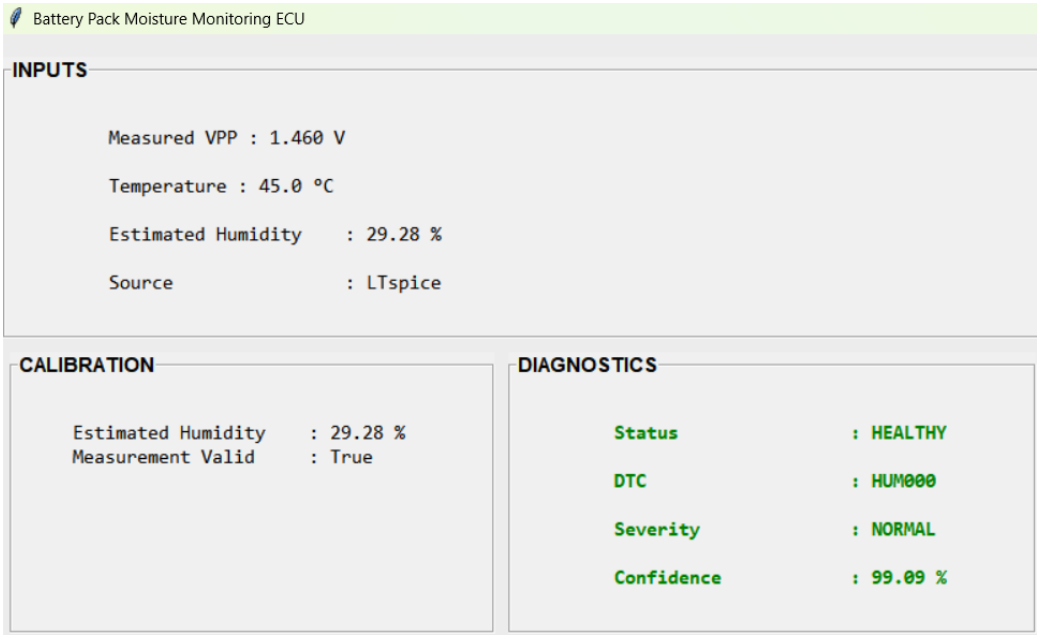


Figure 9.3 — GUI dashboard under nominal (healthy) operation

The GUI dashboard (battery_pack_monitor.py) brings every module onto one screen: measured/calibrated values, diagnostics, battery assessment, thermal management, predictive maintenance, and an overall status summary, updating live as humidity/temperature change. Figure 9 .3 shows a healthy reading (VPP=1.460V at 45°C, humidity 29.28%): valid measurement, HEALTHY status, DTC HUM000, 99.09% confidence. Fault-condition dashboard response is shown in Chapter 13.

9.14 Software Design Considerations

The ECU was split into independent modules mainly to keep testing/debugging manageable a calibration constant change never risked breaking diagnostics logic. This split also lines up naturally with an eventual AUTOSAR Software Component decomposition (Chapter 16).

10.System Execution

This chapter describes the actual operation of the entire system, from an actuator response to an environmental input.

10.1 Environmental Simulation

Relative humidity and temperature are defined as swept parameters inside LTspice rather than read from a physical enclosure, since no physical enclosure exists. Humidity drives the sensor's capacitance directly; temperature affects both the sensor model and, later, the battery-assessment logic.

10.2 Sensor Modelling

The capacitive sensor model converts the swept RH/T values into a capacitance via C(RH,T) = C0(1 + alpha*RH + beta*(T-T0)); a higher humidity raises the dielectric constant and therefore the capacitance, which is what the front-end ultimately measures.

10.3 Analog Signal Conditioning

The AC-excited sensor signal passes through the charge amplifier (capacitance-to-voltage conversion), the gain stage (amplification), and the low-pass filter (noise reduction) in sequence, leaving a stable voltage that represents the environmental condition and is ready to be exported for software processing.

10.4 Python ECU Processing

The exported VPP value is processed through the full chain described in Chapter 9 temperature compensation, calibration, diagnostics, sensor health/diagnostic-confidence scoring, smart risk assessment, and battery/thermal decision logic — strictly in that order, since later stages depend on earlier ones having already corrected or scored the value.

10.5 Decision Logic and Thermal Management

The decision-logic output is feedback into LTspice as the input to the comparator stages: if it calls for cooling, the cooling comparator switches its MOSFET and energizes the cooling load; if it calls for heating, the heating comparator does the same for the heating load. The two controllers were verified independently before being integrated into the complete signal chain.

10.6 System Feedback

Once an actuator has responded, the environmental condition it was reacting to changes, and that updated condition feeds back into the sensor model for the next cycle — forming a closed loop rather than a single open-loop measurement.

10.7 System Timing

Although the current implementation runs as a simulation rather than on a real-time target, each execution cycle follows the same ordering a periodic embedded task would use: acquisition, analog processing, export, temperature compensation, calibration, diagnostics, risk assessment, decision logic, thermal control, and then the next sampling cycle. A future hardware port would map this same sequence onto a timer interrupt or an RTOS task running at a fixed period.

Project File Inventory

Primary project files behind the results and diagrams in this section:

File	Type	Description
HumiditySensor_AFE_v5.asc	LTspice netlist	Sensor model and analog front-end (Figure 6.1)
BTMS_driver.asc / btms_control.inc	LTspice netlist	BTMS Power Driver Board (Figure 6.2)
battery_pack_monitor.py	Python application	GUI dashboard and ECU pipeline (Chapter 7)
history.csv	Data log	Logged ECU runs, Tables 9.1/11.3 and dashboard screenshots

File	Type	Description
HumiditySensor_AFE_v5.log	SPICE log	Source of Table 9.1's .meas values
This report	Documentation	Consolidated design, results, and evaluation record

11. Results and Evaluation

This chapter presents results captured from the LTspice front-end and the Python ECU. Two items remain open at the time of writing, marked where they occur rather than filled with placeholders: a clean over-current trip waveform, and a Battery Health vs. Humidity sweep (both revisited in Chapter 15).

11.1 LTspice Verification

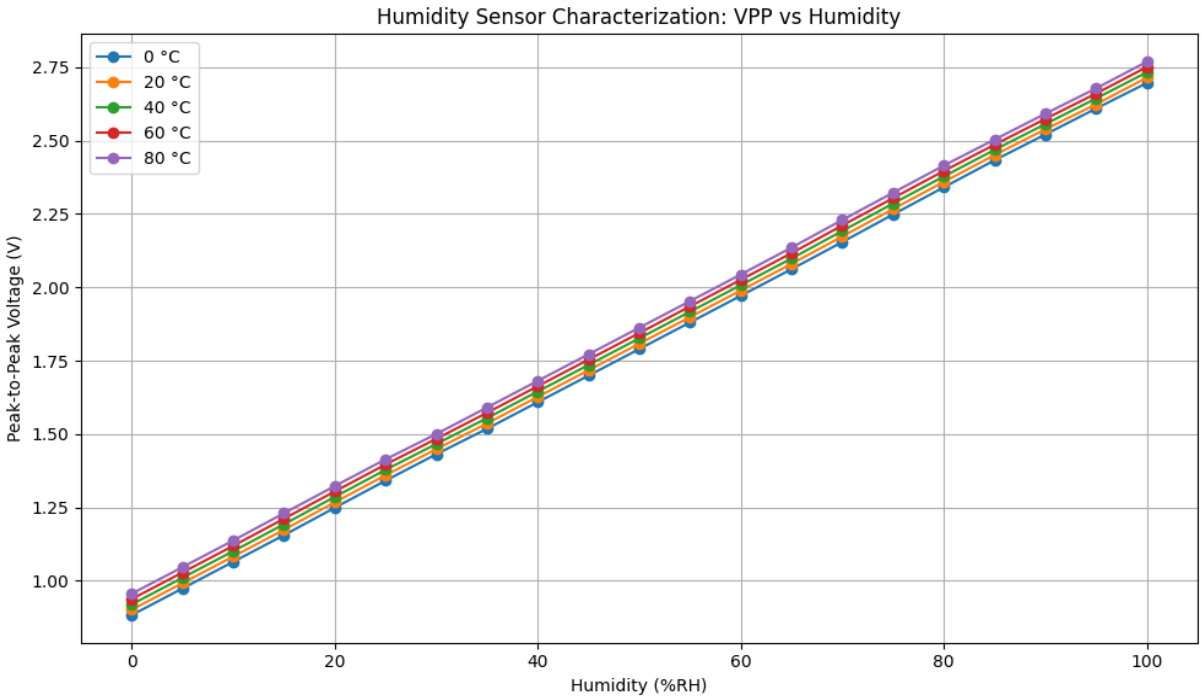


Figure 11.1 — Sensor characterization: measured VPP vs. humidity at five temperatures (0-80 degC)

VPP rises monotonically with humidity at every temperature; the five curves fan out slightly with temperature the raw signature temperature compensation (9.2) is designed to remove.

Front-end stability is confirmed throughout the whole working envelope, not just one tested point, by the waveform's constant shape across all stepped runs, with only amplitude changing. A little portion of the underlying measured log behind Figure 11.1 is shown in Table 11.1.

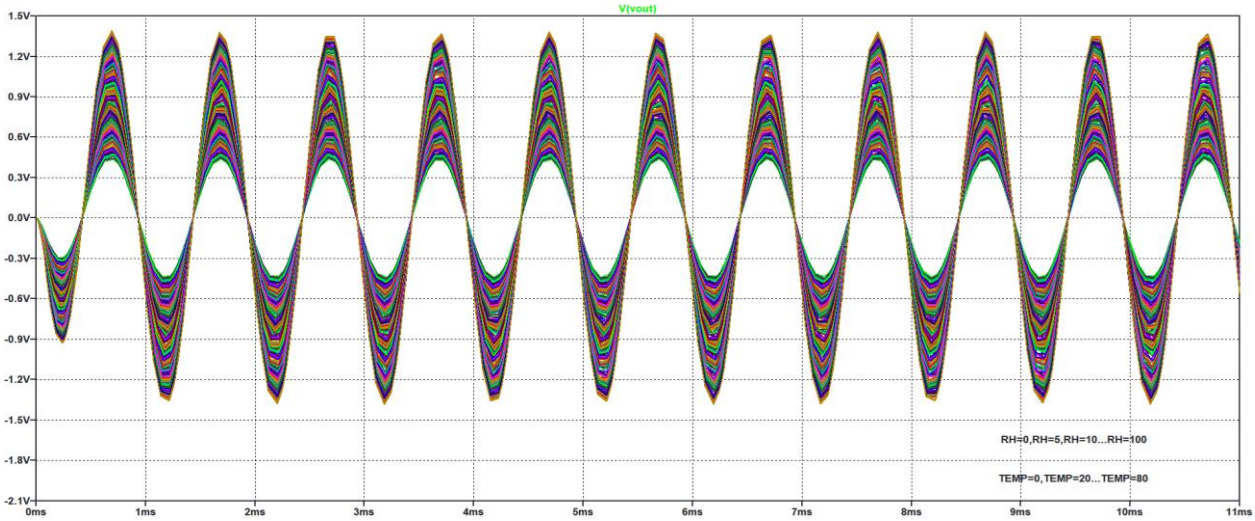


Figure 11.2 — LTspice transient output V(vout) across the full RH x temperature step sweep.

Step	RH (%)	Measured VPP (V)
1	0	0.8837
2	5	0.9741
3	10	1.0648
4	15	1.1567
5	20	1.2490
21	100	2.7033

Table 11.1 — Sample. measured log excerpt (TEMP = 0 degC sweep)

11.2 BTMS Hardware Verification

The cooling and heating drivers were exercised on either side of their thresholds. Figure 11.3 (RH=40%, TEMP=5 degC, below the 15 degC heating threshold) confirms HEAT_ENABLE asserted (~4.95 V). Figure 11.4 (RH=95%, TEMP=38 degC, above 35 degC) confirms COOL_ENABLE asserted.

For the 1 kOhm loads in Table 7.1, the indicator-LED current traces next to these gate voltages read in the picoamp range, which is physically meaningless because the LED branch was probed instead of the main load path. Before being quoted as load-current evidence, the measurements should be taken directly across the load resistor. Since both stages switch at the appropriate threshold, the gate-voltage confirmation is sound in and of itself.

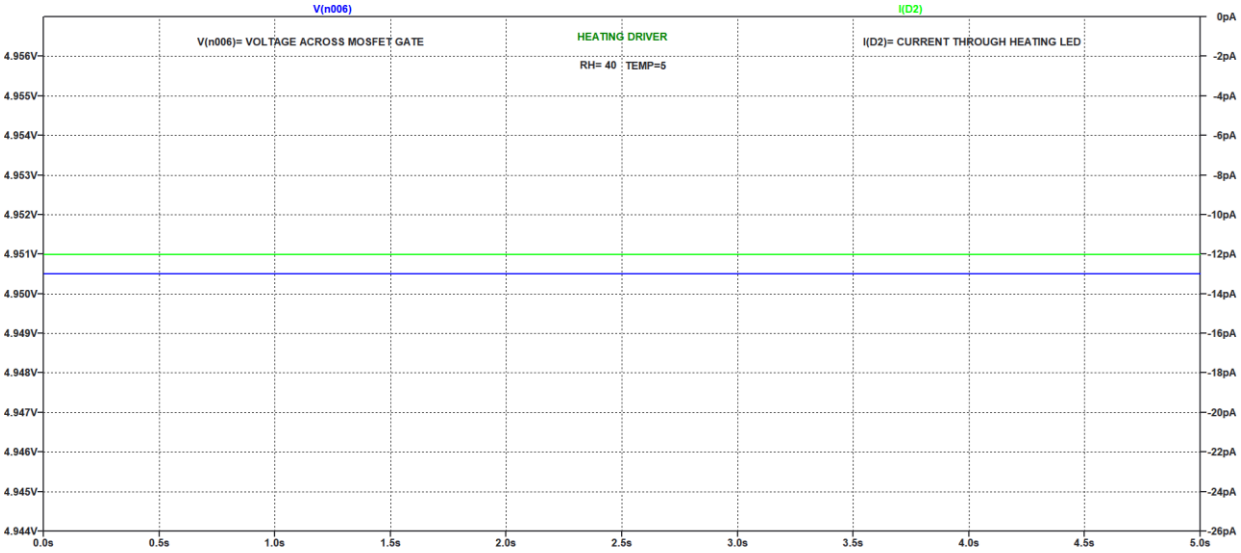


Figure 11.3 — Heating driver: gate voltage confirms HEAT_ENABLE asserted at RH=40%, TEMP=5 degC

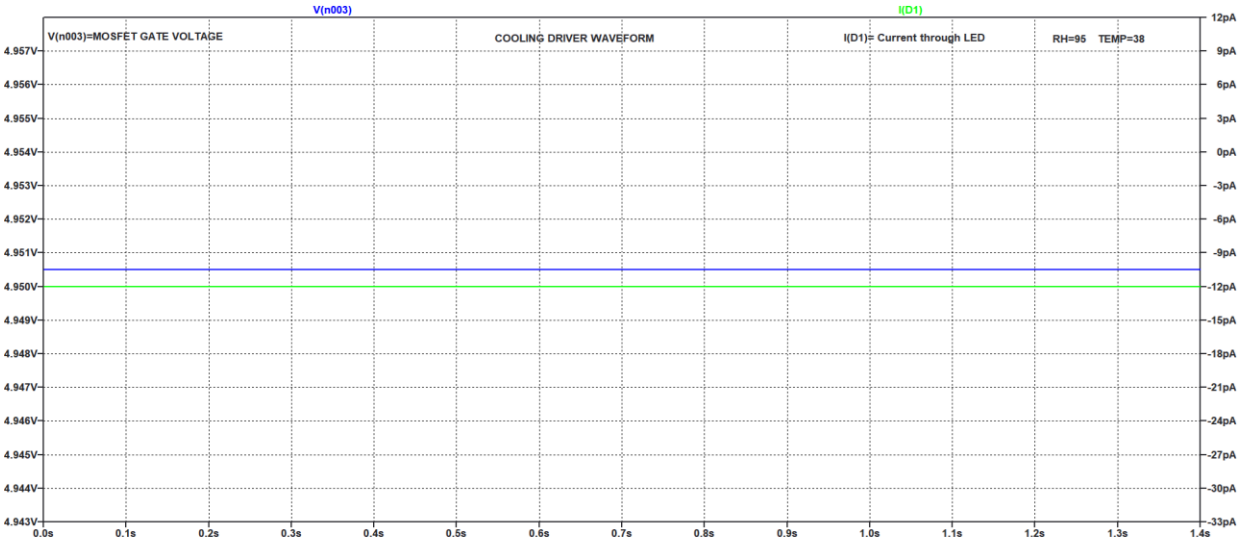


Figure 11.4 — Cooling driver: gate voltage confirms COOL_ENABLE asserted at RH=95%, TEMP=38 degC

11.3 Python Calibration and Compensation Verification

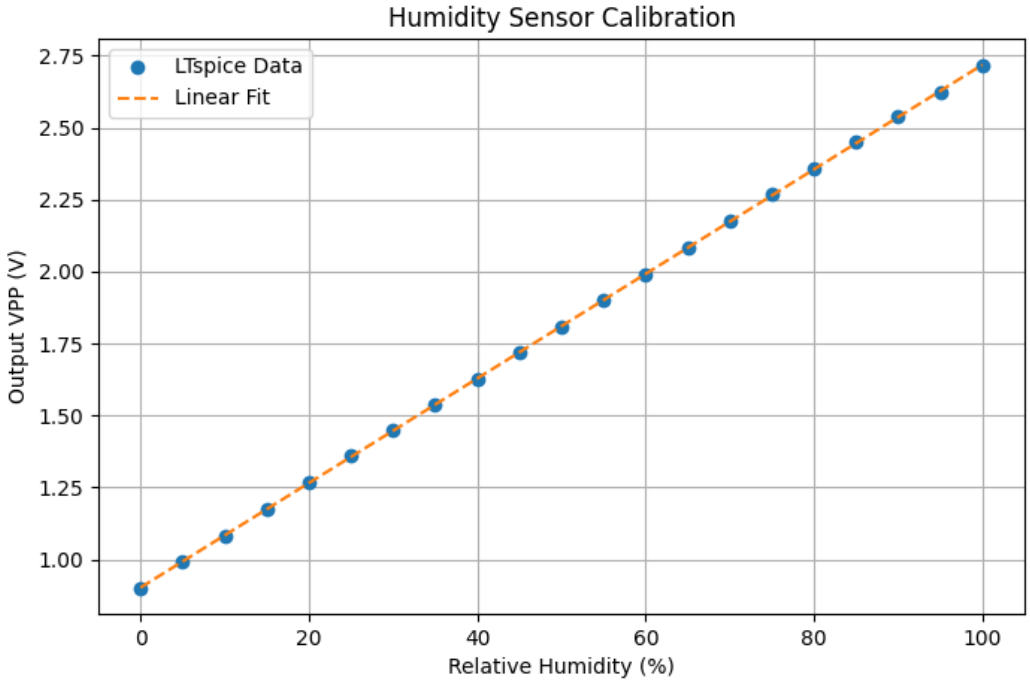


Figure 11.5 — Calibration curve: measured VPP vs. relative humidity, with linear fit

Close agreement between measured points and the fitted line confirms the linear calibration model (8.3) fits well across the full 0-100% RH range, not only where it may have been tuned.

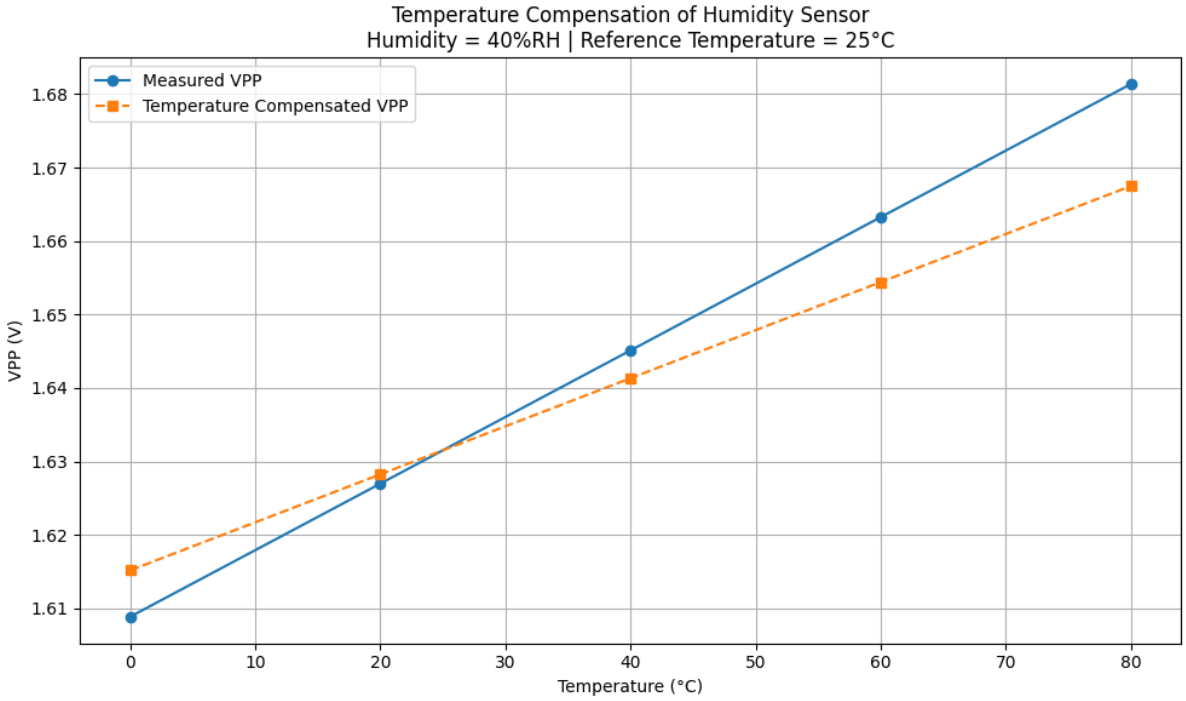


Figure 11.6 — Temperature compensation at fixed 40% RH: measured VPP vs. temperature-compensated VPP

The raw observed VPP rises with temperature at fixed 40% RH between 0 and 80 degrees Celsius, as predicted; the adjusted curve is noticeably flatter, indicating that the compensation term (9.2) eliminates the majority of the temperature-driven offset. The dashboard provides a second, independent confirmation: the same VPP (1.460 V) reads 30.28% RH at 25°C but 29.28% RH at 45°C—a little, accurate downward correction made end-to-end using the actual program.

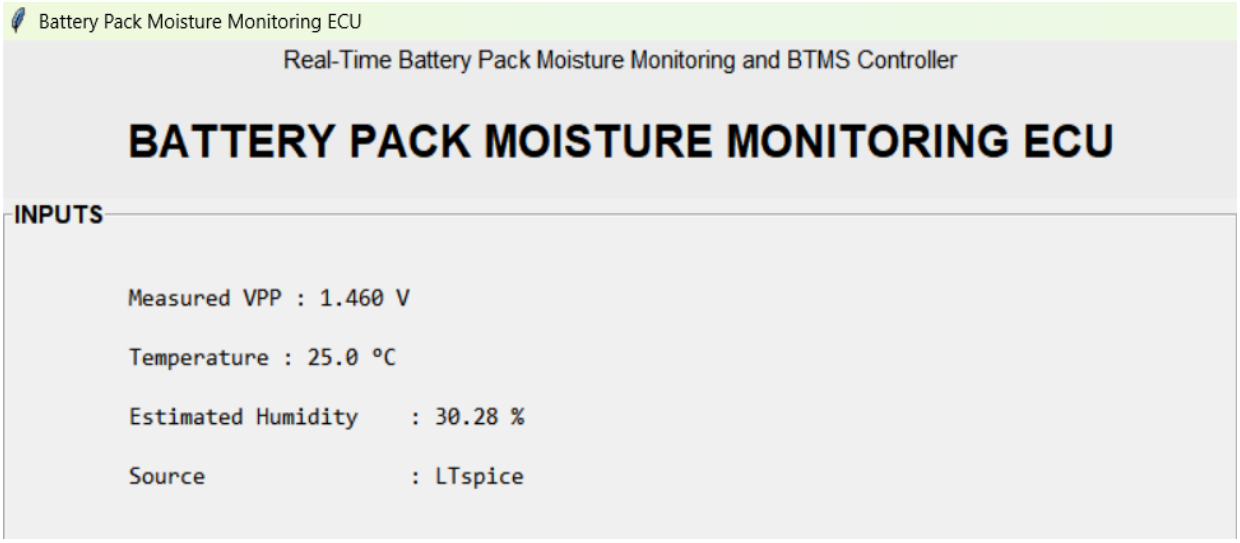


Figure 11.7a — Dashboard reading at 25 degC (reference temperature): VPP 1.460 V -> 30.28% RH

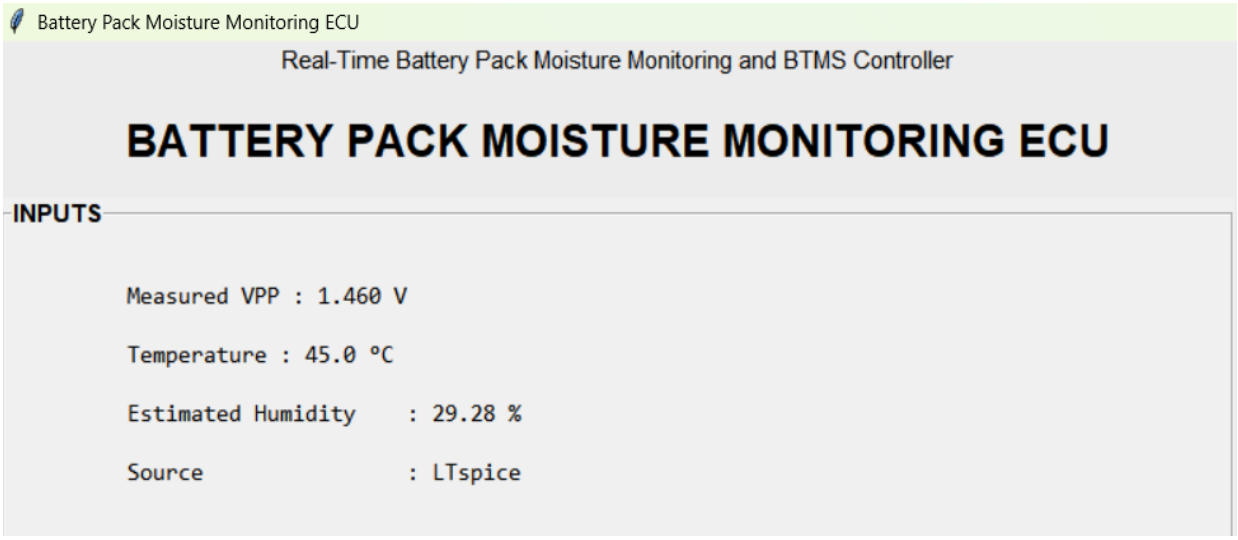


Figure 11.7b — Dashboard reading at 45 degC, same VPP: 1.460 V -> 29.28% RH

11.4 Battery Assessment and Diagnostics Verification

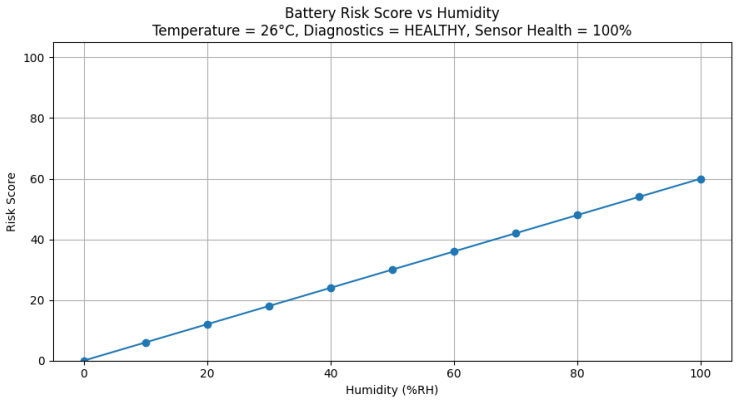


Figure 11.8 — Smart Risk Score vs. Humidity (Temperature=26 degC, Diagnostics=HEALTHY, Sensor Health=100%)

With every other risk factor at its healthy baseline, the score rises smoothly from 0 to 60 across the RH sweep, consistent with the 0.40 humidity weight (8.7) dominating under otherwise-healthy conditions.

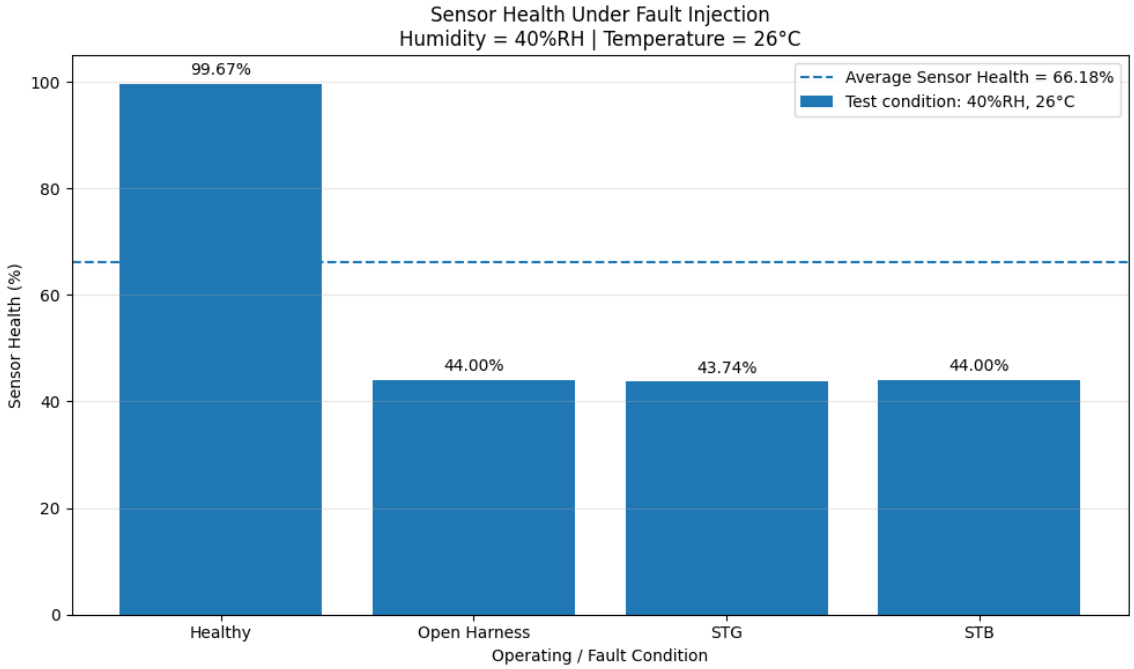


Figure 11.9 — Sensor Health under each fault condition (Humidity=40%RH, Temperature=26 degC)

Healthy scores 99.67%; all three injected faults collapse to ~44%, clustering tightly (43.74-44.00%) rather than spreading — the current SHI penalizes any fault's presence more strongly than it distinguishes fault type (design observation, Chapter 13).

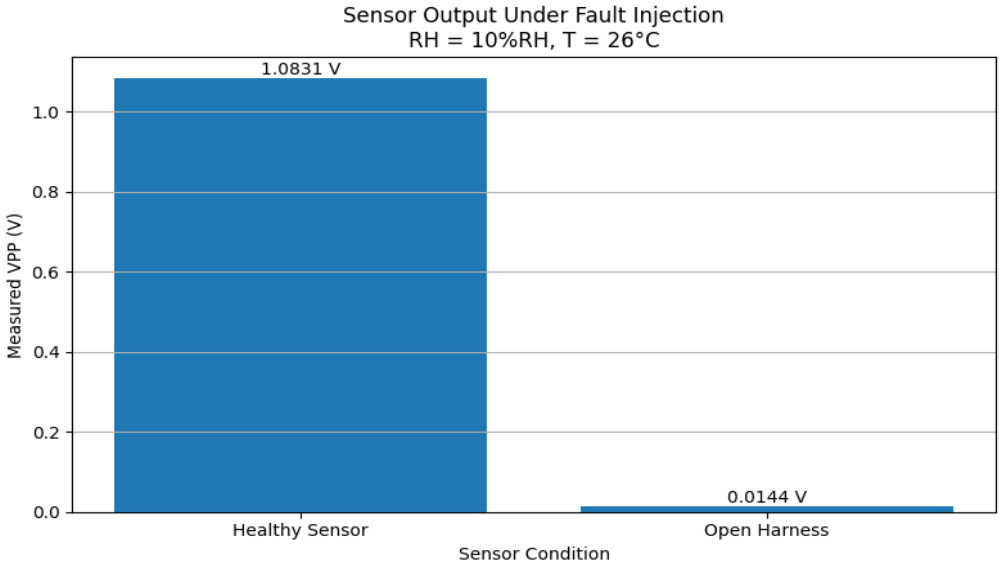


Figure 11.10 — Measured VPP: healthy sensor vs. open-harness fault (RH=10%RH, T=26 degC)

The signal-level cause: a healthy reading of 1.0831 V collapses to 0.0144 V under the open-harness fault — a two-order-of-magnitude drop the diagnostics range check (8.6) is designed to catch.

11.5 Verification Summary

Table 11.2 distinguishes what has been directly confirmed from what remains open.

Module	Check performed	Result
Humidity sensor model	VPP vs. swept RH, 5 temperatures	Confirmed — Fig 9.1
Analog front-end	Transient stability, full sweep	Confirmed — Fig 9.2
Cooling driver	Gate voltage, TEMP=38 (>35)	Confirmed — Fig 9.4
Heating driver	Gate voltage, TEMP=5 (<15)	Confirmed — Fig 9.3
Load current	Direct load-current measurement	Open — LED branch probed, 9.2
Over-current protection	Trip at ~0.5 A	Open — no clean capture, Ch.13
Calibration	Measured VPP vs. fit, full range	Confirmed — Fig 9.5
Temperature compensation	Drift removal across T sweep	Confirmed — Fig 9.6, 9.7
Diagnostics	Fault-induced VPP collapse	Confirmed — Fig 9.10
Sensor Health Index	Healthy vs. 3 fault types	Confirmed, with caveat — Fig 9.9
Smart risk score	RH sweep, healthy baseline	Confirmed — Fig 9.8
Battery Health vs. Humidity	Dedicated sweep plot	Open — not captured, Ch.13

Table 11.2 — Verification summary (confirmed vs. open items)

This chapter documents the concrete problems encountered while moving through the development phases in Chapter 5, and the solution adopted for each. These are the parts of the project that did not work on the first attempt.

12.Engineering Challenges and Solutions

12.1 Sensor Modelling Instability

Problem: Early attempts to represent humidity as a variable capacitor produced unstable transient results, traced to parameter definitions allowing inconsistent capacitance values between steps.

Solution: A single parameterized capacitance equation was introduced so humidity and temperature vary systematically and consistently across a run.

12.2 Low Front-End Output Amplitude

Problem: The first amplifier design produced only a small output swing, making humidity levels hard to distinguish at the output.

Solution: The gain stage was redesigned and component values adjusted until the output amplitude clearly resolved the intended range.

12.3 Non-Linear Calibration Error

Problem: Treating output voltage as directly proportional to humidity produced significant errors, since the real sensor response is non-linear.

Solution: A polynomial/linear calibration was implemented in Python, fitted against known voltage/humidity pairs rather than assumed proportional.

12.4 Temperature-Dependent Drift

Problem: Humidity estimates shifted with ambient temperature even at constant true humidity, since the sensor response is itself temperature-dependent.

Solution: A temperature-compensation step was added immediately after calibration, removing most of the temperature-driven offset across the tested range.

12.5 Comparator Polarity and Threshold Tuning

Problem: The thermal comparators initially switched in the wrong direction, so cooling/heating triggered under the opposite intended conditions.

Solution: Comparator inputs were swapped and thresholds re-tuned until both paths switched at the correct condition.

12.6 MOSFET Switching Reliability

Problem: The generic LTspice NMOS model did not switch reliably once the simulated load was increased to a realistic level.

Solution: The generic model was replaced with an IRFZ44N model, and load resistance adjusted for consistent, repeatable switching.

12.7 Hardware/Software Integration

Problem: LTspice and Python initially ran as two disconnected pieces of work, with no defined path between them.

Solution: A CSV export from LTspice, read automatically by Python, established a repeatable hardware-to-software data path resembling a real ECU receiving digitized sensor data.

12.8 Deriving Trustworthy Calibration Constants

Problem: An early calibration used assumed offset/sensitivity values rather than ones tied to the actual front-end, producing estimates that drifted at other operating points.

Solution: The offset (0.8677) and sensitivity (0.0061) constants (Section 8.3) were instead fitted from VPP values measured at several known humidity levels during LTspice characterization.

12.9 Choosing Weights for the Smart Risk Score

Problem: Combining five factors into one risk score raised an immediate question: how much should each count, and how can that be justified rather than arbitrary?

Solution: Humidity got the largest weight (0.40, most directly tied to moisture ingress/corrosion), temperature next (0.20), and sensor health/diagnostic confidence/thermal condition the remainder (0.15/0.15/0.10), summing to 1.0. Documented explicitly as engineering assumptions, open to revision with future evaluation data.

13. Verification, Validation and Test Plan

13.1 Verification Strategy

Verification asks whether the system was built correctly. Each subsystem listed below was checked individually before being integrated with the rest: the sensor model, the analog front-end, calibration, temperature compensation, diagnostics, the smart risk-scoring engine, predictive maintenance, and the comparator/MOSFET BTMS controller stages.

13.2 Validation Strategy

Validation asks whether the system meets its intended objective, checked against the objectives listed in Section 3.4. Humidity sensing, VPP-based signal conditioning, calibration, temperature compensation, fault detection, risk-based battery assessment, predictive-maintenance estimation, and thermal-management response were each evaluated

individually against that objective list, with the outcome recorded in the traceability matrix below.

13.3 Requirement Traceability Matrix

Requirement	Implementation	Verification method
Humidity sensing	Capacitive sensor model	LTspice simulation
Signal conditioning	Analog Front-end (VPP)	Waveform inspection
Temperature compensation	Compensation module	Multi-temperature simulation runs
Calibration	Linear fit (Python)	Recovered-vs-known RH comparison
Diagnostics	Fault-detection module	Artificial fault injection
Battery risk assessment	Smart risk score, protection stages	Threshold-crossing test across TC1–TC8
Predictive maintenance	Trend/health-index module	Trend-injection test
Thermal management	Comparator + MOSFET BTMS stages	Threshold-crossing test
Sensor redundancy / functional-safety state machine	Not yet implemented	Deferred — see Chapter 15

Table 13.1 — Requirement traceability matrix

13.4 Test Cases

TC1-TC6 below illustrate the intended input/expected-output pattern for each scenario; these remain representative of the design targets in Chapters 8-9 pending a full logged run. TC7 has been split into its three constituent fault sub-cases and populated with real logged values, since all three were actually exercised and captured (Section 11.5).

Test	Temp.	Humidity	Expected outcome
TC1	25 degC	40%	Healthy, Standby
TC2	38 degC	65%	Healthy, Cooling (low)
TC3	45 degC	92%	Critical, Cooling (high), Protection Stage 4
TC4	10 degC	40%	Heating mode
TC5	38 degC	95%	Critical, Immediate Inspection recommended
TC6	38 degC	95% (degraded sensor health)	Higher risk score, maintenance recommendation raised
TC7a	26 degC	open harness fault	See Table 12.3 — logged result
TC7b	30 degC	short-to-ground fault	See Table 12.3 — logged result
TC7c	34 degC	short-to-battery fault	See Table 12.3 — logged result

Test	Temp.	Humidity	Expected outcome
TC8	over-current in BTMS (LTspice)	—	Protection comparator trips — not yet captured (Chapter 13)

Table 13.2 — Demo test case matrix (TC1-TC6, TC8 illustrative; TC7a-c logged, see Table 13.3)

13.5 Fault Injection — Logged Results

All three fault sub-cases were run against the real Python ECU and logged automatically. Table 13.3 reproduces the logged values; Figures 13.1-13.3 show the corresponding dashboard state for each.

Sub-case	Temp.	Measured VPP	Diagnostic Status	DTC	Confidence	Sensor Health	Protection Stage
TC7a — Open harness	26 degC	0.0144 V	OPEN_HARNESS	HUM001	99.98%	43.99%	Stage 4 / Emergency
TC7b — Short to ground	30 degC	0.0013 V	SHORT_GROUND	HUM002	99.33%	43.71%	Stage 4 / Emergency
TC7c — Short to battery	34 degC	0.000 V	SHORT_BATTERY	HUM003	100.0%	44.00%	Stage 4 / Emergency

Table 13.3 — Logged fault-injection results (source: history.csv, real ECU runs)

All three fault types are correctly classified with high confidence (>99%), correctly escalate to Protection Stage 4 with charging disabled, and correctly collapse Sensor Health to approximately the same 44% floor observed in Figure 10.9 — consistent behaviour between an isolated diagnostic-module test and a full end-to-end logged run.

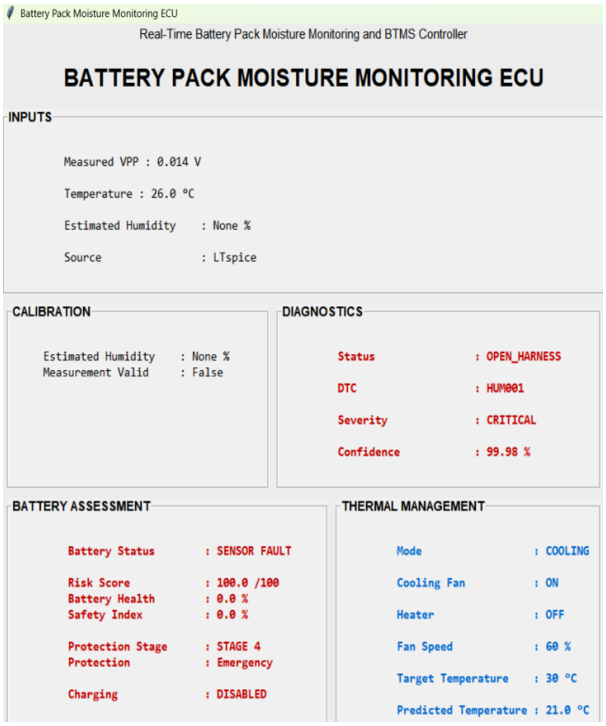


Figure 13.1 — Dashboard under TC7a (open harness fault)

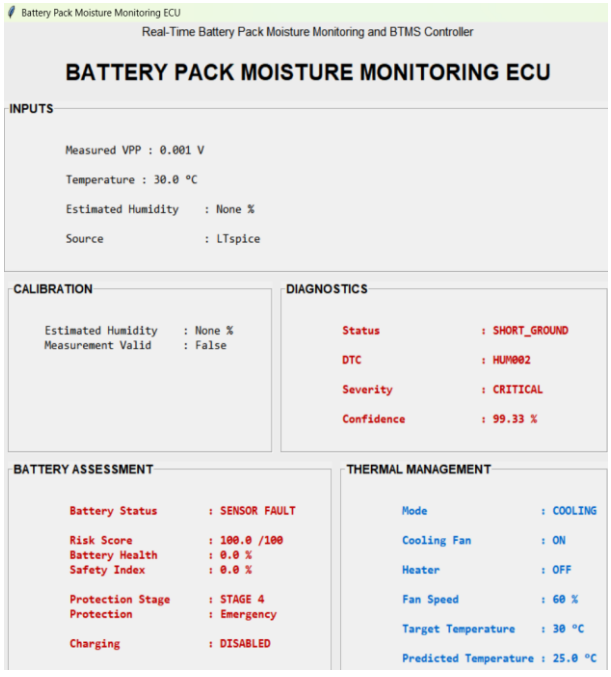


Figure 13.2 — Dashboard under TC7b (short-to-ground fault)

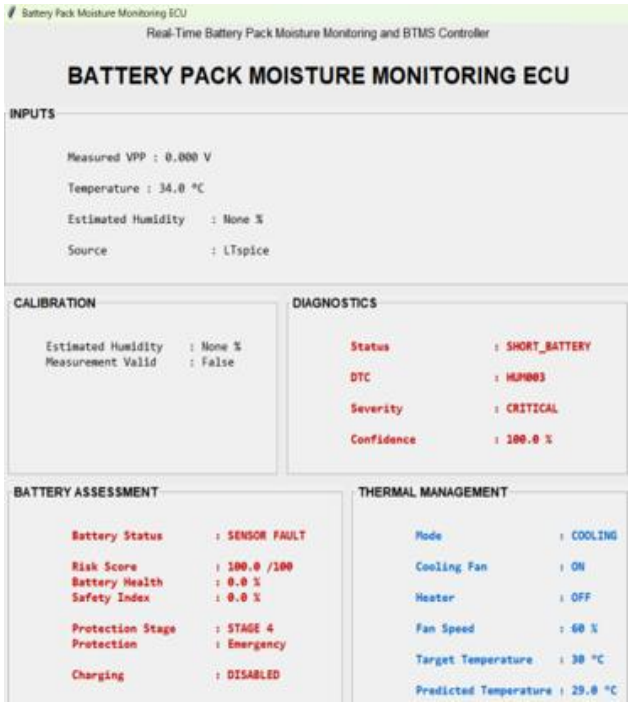


Figure 13.3 — Dashboard under TC7c (short-to-battery fault)

14. Disscusion and Design Rationale

14.1 Tool Choices: LTspice and Python

LTspice was chosen over Proteus, Multisim, or MATLAB Simscape for its accurate analog behavioural models, flexible parameter sweeping, efficient transient/AC analysis, and free availability — well suited to a project built around iterative simulation rather than a fixed hardware target. Python was chosen for the ECU layer over MATLAB or embedded C mainly for development speed: its numerical/data-handling libraries, straightforward CSV I/O, and fast iteration suited a software design still evolving through most of the phases in Chapter 5.

14.2 Hardware/Software Partitioning

LTspice owns everything physically analog (sensor, front-end, comparators, MOSFET drivers); Python owns everything that is a decision or calculation (compensation, calibration, diagnostics, risk scoring, predictive maintenance, thermal/battery decision logic) — mirroring how these responsibilities typically divide between analog hardware and an automotive ECU.

14.3 Comparison with a Conventional BMS

Table 14.1 compares this project's scope against a conventional battery management system.

Feature	Conventional BMS	This system
Temperature monitoring	Yes	Yes
Humidity monitoring	Rare	Yes

Feature	Conventional BMS	This system
Calibration / compensation	Limited	Yes
Fault diagnostics	Basic	Range/spike/stuck/drift + Sensor Health Index
Risk-based protection staging	Limited	Yes (weighted risk score, 4 protection stages)
Predictive maintenance	Rare	Yes
Thermal actuation	Optional	Comparator + MOSFET BTMS controller (simulated)
Hardware/software co-design	Not typical	Yes

Table 14.1 — Comparison with a conventional BMS (this system does not perform cell balancing, SOC, or SOH estimation, and is not a full BMS replacement)

Compared with a bare humidity-sensor readout, this project adds automated compensation/calibration, diagnostic fault detection with a sensor health index, a weighted risk-scoring engine feeding staged protection actions, trend-based predictive maintenance, and a comparator-driven BTMS actuation stage — turning a single measurement into a small closed-loop response system, even though every part currently exists only in simulation.

14.4 Observation: Sensor Health Index Does Not Distinguish Fault Types

Figure 10.9 showed all three injected fault types collapsing Sensor Health to a tightly clustered 43.7-44.0%, rather than three distinct values. This is arguably correct for the score's purpose — SHI answers "can this reading be trusted now", and the honest answer is no for all three — but it means SHI alone cannot distinguish which fault occurred; that comes entirely from the diagnostic status/DTC fields (Table 12.3), which do differ correctly (HUM001/002/003). Noted as a design observation, not a defect: SHI is a trust score, not a fault classifier, and should not be read as one.

15. Limitation and Risk Assessment

15.1 Limitations

The entire system exists only in simulation: no sensor, circuit, or actuator has been built or exposed to a real environment, so Chapter 10 describes expected simulated behaviour, not measured physical performance. The battery model is simplified (no cell balancing, SOC, or SOH), so the system is a moisture/thermal-monitoring addition to a BMS, not a replacement. Comparator/MOSFET behaviour uses generic/substituted SPICE models rather than characterized commercial parts, and the Python side assumes an already-digitized signal without modelling ADC quantization, sampling-rate, or bus effects.

Two functional gaps are worth stating plainly: there is no sensor redundancy (a single measurement path, so a fault defeating diagnostics has no second sensor to cross-check), and the risk-scoring/protection-stage logic is not a formally defined functional-safety state

machine — a weighted decision system, not a verifiable set of safety states with a proven fallback. Both were intentionally deferred (Chapter 14).

Three narrower gaps surfaced while assembling Chapter 9's results: the BTMS driver captures (Figs 10.3/10.4) confirm gate-voltage switching but probed an indicator-LED branch, not main load current; no clean over-current trip capture was available (Section 7.4.3 verified only by schematic inspection); and a dedicated Battery Health vs. Humidity sweep was not available and was not fabricated. All three are concrete near-term actions in Chapter 14.

15.2 Risk Assessment

Risk	Impact	Mitigation
Sensor model drift	Medium	Calibration and compensation stages
Signal noise	Medium	Low-pass filter stage
Comparator instability	Medium	Threshold tuning (Chapter 10)
MOSFET switching reliability	Low	IRFZ44N model substitution
Software logic errors	Medium	Modular design, per-module testing
LTspice convergence issues	Medium	Parameter/topology adjustment during development

Table 15.1 — Technical risk register

Development-level risks — an incorrect mathematical model, simulation convergence problems, hardware/software synchronization, calibration accuracy, and the simplified battery model — were addressed progressively through the iterative phases described in Chapter 4, rather than resolved up front.

16.Future Work and Industrial Relevance

16.1 Near-Term Priorities

Three concrete measurement tasks should precede the next revision: re-probe cooling/heating load current directly across the load (Figure 7.2) rather than the LED branch; capture a clean over-current trip waveform (Section 7.4.3) by forcing load current past ~0.5 A; and run a dedicated Battery Health vs. Humidity sweep (mirroring Figure 10.8). None require new design work, only additional captures.

Two larger items follow: sensor redundancy (a second, independent humidity path to cross-check the primary sensor) and a formally defined functional-safety state machine (verifiable OK/DEGRADED/FAULT/CRITICAL states with a proven fallback, distinct from the risk-scoring logic already implemented). Both were intentionally scoped out and are the most natural next additions.

16.2 Path to Physical Hardware and AUTOSAR

The LTspice front-end could be realized with a commercial capacitive sensor and discrete/integrated charge-amplifier circuitry; the Python ECU could be ported to embedded C on an automotive microcontroller; and the BTMS controller could gain PWM-driven variable-speed cooling instead of on/off switching. The module boundaries already drawn in Chapter 8 map fairly directly onto AUTOSAR Software Components, with CAN or Automotive Ethernet replacing today's direct Python function calls — a deliberate design choice from early in the project.

16.3 Further Work

The predictive-maintenance trend estimate could use a learned model instead of a fixed slope; the risk score's weights (8.7) could be tuned against real labelled data; and battery assessment could incorporate additional signals (e.g. pressure) if scope expands. Beyond EV battery packs, the same architecture applies to windshield fog prediction, cabin HVAC humidity control, ECU-enclosure moisture monitoring, and stationary battery storage anywhere a sealed enclosure needs humidity awareness and a thermal response, not just temperature.

17. Conclusion

This project set out to show that a simulation-only platform, combining LTspice's analog modelling with a Python-based ECU, can emulate an automotive battery-pack moisture and thermal monitoring system without a physical build. The resulting BTMS models the full chain from a capacitive sensor through VPP-based conditioning, compensation, calibration, diagnostics, a weighted smart risk score, and predictive maintenance, down to a comparator/MOSFET BTMS controller with over-current protection. Against the objectives in Section 2.4, the system is ~95% complete, with sensor redundancy and a formal functional-safety state machine as the two deferred items.

The main contribution is less any single module and more that the hardware and software halves were developed together and kept synchronized — each phase in Chapter 4 only moved forward once verified, and the engineering problems in Chapter 11 were the direct result of that boundary being exercised for the first time, not incidental bugs. This is also why the architecture, more than any single numeric result, is what is most directly reusable in a future hardware build or AUTOSAR port. The clearest next step is closing the gap to a physical prototype, starting with the analog front-end, where a real sensor's non-idealities would immediately test the calibration and compensation assumptions made here in simulation.

Bibliography

The following list covers the tools, standards, and general concepts drawn on throughout this report. Course-specific literature and any papers assigned or consulted as part of the M.Sc. programme should be added by the author, since they are not represented here.

[1] Analog Devices Inc., Ltspice Free SPICE Simulation Software, https://www.analog.com/en/design-center/design-tools-and-calculators/ltspice-simulator.html

[2] Python Software Foundation, Python Language Reference, https://www.python.org

[3] Harris, C.R. et al., "Array programming with NumPy," Nature, 2020.

[4] McKinney, W., "Data Structures for Statistical Computing in Python," Proc. of the 9th Python in Science Conference, 2010 (pandas).

[5] International Organization for Standardization, ISO 26262 — Road Vehicles: Functional Safety (concept reference).

[6] AUTOSAR Consortium, AUTOSAR Classic Platform Specifications (concept reference), https://www.autosar.org

[7] Vishay Semiconductors / generic capacitive humidity sensor application notes (dielectric-constant-based sensing principle).

[8] Infineon Technologies, IRFZ44N Power MOSFET Datasheet (device model reference).

Appendix A

Abbreviations:

Abbreviation	Description
AFE	Analog Front-End
ADC	Analog-to-Digital Converter
AUTOSAR	Automotive Open System Architecture
BMS	Battery Management System
BTMS	Battery Pack Moisture Monitoring and Thermal Management System
ECU	Electronic Control Unit
EV	Electric Vehicle
HI	Health Index
LPF	Low-Pass Filter
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
RH	Relative Humidity
SOC / SOH	State of Charge / State of Health

Appendix B

Symbols:

Symbol	Meaning
RH	Relative humidity
T	Temperature (runtime input, not constant)
Tref / T0	Reference temperature (25 degC, constant)
C	Capacitance
C0	Nominal (reference) sensor capacitance
V	Voltage
VPP, VMAX, VMIN	Peak-to-peak, maximum, and minimum output voltage
VPP_corr	Temperature-compensated peak-to-peak voltage
alpha	Humidity sensitivity coefficient (sensor model)
beta	Temperature sensitivity coefficient (sensor model)
k	Temperature-compensation coefficient (0.0002525 V/degC)
SHI	Sensor Health Index
DC	Diagnostic Confidence
HI	Health index

Appendix C

Complete Equations Summary

This table consolidates every equation referenced in Chapter 9, marking which are directly implemented in code and which are threshold-based decision logic described here in equation-like shorthand for convenience.

Feature	Equation / logic	Type
Peak-to-peak voltage	VPP = VMAX - VMIN	Implemented
Temperature compensation	VPP_corr = VPP - 0.0002525(T - 25)	Implemented
Calibration	RH = (VPP_corr - 0.8677) / 0.0061	Implemented
Sensor Health Index	SHI = 100 - Penalty	Algorithmic (decision logic)
Diagnostic Confidence	Confidence = 100 - Error	Algorithmic (decision logic)
Smart Risk Score	Risk = 0.40H + 0.20T + 0.15SHI + 0.15DC + 0.10TM	Implemented (weights are engineering assumptions)
Battery Health	BatteryHealth = 100 - 0.5 x RiskScore	Implemented
Safety Index	SafetyIndex = 100 - RiskScore	Implemented
Moisture / Corrosion / Condensation Risk	Threshold bands on RH and T	Algorithmic (decision logic)
Protection Stage	Threshold bands on Risk Score	Algorithmic (decision logic)
Thermal Management	T<15 Heating; 15–35 Standby; T>35 Cooling	Algorithmic (decision logic)
Current-sense / over-current trip	V = I x R; trip at ≈0.05 V ⇒ ≈0.5 A	Implemented (hardware, Ohm's law)